

ESpect Real Packet Capture Startup Guide

Version: 1.0

Date: 2026-09-18

This guide starts ESpect from a clean state, creates a real IPsec packet capture, validates the PCAP, copies it to Windows, and sends it to the analyzer.

1. System layout

Component	Where it runs	Address or URL	Purpose
Windows host	Windows PowerShell	C:\Users\Subhadeep\ESpect	Frontend, backend, and analysis
VM1	Linux terminal	192.168.160.128	Controller, traffic coordinator, and capture endpoint
VM2	Linux terminal	192.168.160.129	Second IPsec endpoint and traffic peer

Both VM agents listen on TCP port 9000. VM1's controller connects to both agents. The browser is not the packet source: VM1's `packet_capture.py` starts the real `tcpdump` capture and generates traffic through the two VMs.

Expected paths:

Windows: C:\Users\Subhadeep\ESpect
VMs: /home/kali/ESpect

2. Required software

Windows host

- Python and `ipsec_analyzer\backend\.venv`
- Node.js and npm
- The repository at C:\Users\Subhadeep\ESpect
- VMware or another hypervisor with both Linux VMs

Both Linux VMs

- Python 3 and PyYAML
- `tcpdump`
- strongSwan with `swanctl`
- sudo access without a password prompt while capture runs
- The repository copied to /home/kali/ESpect
- Experiment interface `eth1`

The current capture code uses `eth1` as `CAPTURE_INTERFACE`. Check it before the first run:

```
ip -br addr
ip -br link
```

If the experiment network is not `eth1`, update the interface in `testbed/sih-ipsec-analyzer_vm1/scripts/packet_capture.py` before running.

3. First-time VM preparation

Perform once on each VM:

```
cd /home/kali/ESpect
sudo apt update
sudo apt install -y python3 python3-pip python3-venv tcpdump strongswan strongswan-swanctl
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install pyyaml
```

Verify the tools:

```
python3 --version
tcpdump --version
swanctl --version
```

Verify the local import from each VM project root. On VM1:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1
export PYTHONPATH=$PWD
python3 -c "import src.agent; print('agent import OK')"
```

On VM2, replace the final directory:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm2
export PYTHONPATH=$PWD
python3 -c "import src.agent; print('agent import OK')"
```

The `PYTHONPATH` line is required because `run_agent.py` imports the local `src` package.

4. Start strongSwan on both VMs

Run on VM1 and VM2:

```
sudo -v
sudo systemctl start strongswan
sudo systemctl is-active strongswan
which swanctl
swanctl --list-conns
```

The service check must print `active`. If the service name differs:

```
systemctl list-units --type=service | grep -i strong
```

Do not continue until `swanctl` is installed and the service can be started.

5. Verify the VM network

On VM1:

```
ip -br addr
ping -c 1 192.168.160.129
```

On VM2:

```
ip -br addr
```

```
ping -c 1 192.168.160.128
```

Ping must succeed in both directions. Expected addresses are VM1 **192.168.160.128** and VM2 **192.168.160.129**.

6. Start the agents

Open a dedicated terminal in VM2 and leave it running:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm2
export PYTHONPATH=$PWD
sudo -v
python3 scripts/run_agent.py
```

Open a dedicated terminal in VM1 and leave it running:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1
export PYTHONPATH=$PWD
sudo -v
python3 scripts/run_agent.py
```

Both commands stay in the foreground and normally print no continuous output. Do not close either terminal or press **Ctrl+C**.

Confirm both ports from VM1:

```
nc -zv -w 2 192.168.160.128 9000
nc -zv -w 2 192.168.160.129 9000
```

If **nc** is unavailable:

```
timeout 2 bash -c '</dev/tcp/192.168.160.128/9000' && echo VM1-agent-OK
timeout 2 bash -c '</dev/tcp/192.168.160.129/9000' && echo VM2-agent-OK
```

7. Start the Windows backend

Open PowerShell and check port **8000**:

```
Get-NetTCPConnection -LocalPort 8000 -State Listen -ErrorAction SilentlyContinue | Select-Object LocalAddress,LocalPort,Owner
```

If an old process is listening and it is safe to stop, use its displayed PID:

```
Stop-Process -Id <PID> -Force
```

Start the backend and keep this window open:

```
Set-Location 'C:\Users\Subhadeep\ESpect\ipsec_analyzer\backend'
.\venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Verify it from another PowerShell window. Use **curl.exe**, not plain **curl**:

```
curl.exe -sS --max-time 5 http://127.0.0.1:8000/health
```

Expected response:

```
{"status":"ok"}
```

8. Start the Windows frontend

Open another PowerShell window and keep it open:

```
Set-Location 'C:\Users\Subhadeep\ESpect\ipsec_analyzer\frontend'  
npm install  
npm run dev -- --host 127.0.0.1 --port=5173
```

Open:

```
http://127.0.0.1:5173
```

If Vite chooses another port, use the URL printed by Vite. The backend currently allows frontend ports **5173** and **5174**.

9. Confirm the capture configuration

On VM1:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1  
cat config/configuration.yaml
```

Expected default configuration:

```
ipsec_mode: transport  
encryption: aes128-gcm16  
integrity: none-aead  
dh_group: modp2048  
pfs: true  
ip_version: ipv4  
traffic_type: voip
```

Supported traffic types are **voip**, **whatsapp**, **email**, **web**, **icmp**, and **video**. Supported modes are **transport** and **tunnel**; supported IP versions are **ipv4** and **ipv6**.

To create a validated configuration with defaults:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1  
export PYTHONPATH=$PWD  
python3 scripts/configurator.py --output config/configuration.yaml
```

10. Check sudo before capture

Run on both VMs:

```
sudo -v  
sudo -n true && echo 'non-interactive sudo OK'
```

This matters because the capture helper starts **tcpdump** with **sudo -n**. If this fails, configure sudo for the lab user before continuing.

11. Run the real packet capture

Run only on VM1, in a new terminal. Keep both agent terminals running:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1  
export PYTHONPATH=$PWD  
sudo -v  
python3 scripts/packet_capture.py config/configuration.yaml 30 --output captures/live_capture.pcap
```

The arguments mean:

Argument	Meaning
<code>config/configuration.yaml</code>	IPsec experiment configuration
<code>30</code>	Capture and traffic duration in seconds
<code>--output captures/live_capture.pcap</code>	PCAP path on VM1

The script performs six stages:

1. Configure both agents.
2. Apply strongSwan IPsec configuration.
3. Start `tcpdump` on `eth1` before IKE negotiation.
4. Initiate IPsec from VM1.
5. Verify both IKE SAs and generate traffic.
6. Stop capture, terminate IPsec, download, and validate the PCAP.

The expected output file is:

```
/home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1/captures/live_capture.pcap
```

12. Validate the PCAP on VM1

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1
ls -lh captures/live_capture.pcap
file captures/live_capture.pcap
tcpdump -nn -r captures/live_capture.pcap | head -20
```

The file must exist and `tcpdump` must print packet records. On failure inspect:

```
cat debug/logs/tcpdump.log
```

For the default IPv4 configuration, the filter is:

```
host 192.168.160.128 and host 192.168.160.129 and (udp port 500 or udp port 4500 or ip proto 50)
```

13. Copy the PCAP to Windows

The analyzer cannot read a file that exists only inside a VM. Copy the PCAP using a hypervisor shared folder, SCP, or another approved transfer method.

Expected Windows destination:

```
C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap
```

Example from PowerShell when SSH is enabled:

```
scp kali@192.168.160.128:/home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1/captures/live_capture.pcap 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
```

Replace `kali` with the actual VM username if different. Verify the Windows file:

```
Test-Path 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
Get-Item 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap' | Select-Object FullName, Length
```

14. Analyze the PCAP

1. Confirm the backend health response is `{"status": "ok"}`.
2. Open `http://127.0.0.1:5173`.
3. Open the analysis/upload view.
4. Select the Windows copy of `live_capture.pcap`.
5. Click `Analyze`.
6. Wait for the result before starting another upload.

The backend accepts `.pcap` and `.pcapng` through `/api/analyze`.

PowerShell API test:

```
$pcap = 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
curl.exe -sS --max-time 120 -F "file=@$pcap" http://127.0.0.1:8000/api/analyze
```

Use a longer timeout for larger captures. To generate the PDF security report:

```
$pcap = 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
curl.exe -sS --max-time 120 -F "file=@$pcap" http://127.0.0.1:8000/api/report -o 'C:\Users\Subhadeep\ESpect\Docs\ESpect_Rep
```

15. Normal shutdown

The capture script exits after its duration. If it is stuck, press `Ctrl+C` in its terminal. After the PCAP is copied, press `Ctrl+C` in each agent terminal, then in the backend and frontend terminals.

If a later run reports a stale IPsec SA, run on both VMs:

```
sudo swanctl --terminate --ike learning-vpn || true
sudo swanctl --list-sas
```

Restart both agents before retrying.

16. Troubleshooting

Placeholder path error

`/path/to/testbed/...` is not a real path. Use:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm2
```

ModuleNotFoundError: No module named 'src'

Run from the project root and set `PYTHONPATH`:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm2
export PYTHONPATH=$PWD
python3 scripts/run_agent.py
```

Cannot communicate with an agent

Check ping, TCP port `9000`, the agent terminal, and any VM firewall. Both agents must be running.

tcpdump exited immediately

Check the interface and privileges:

```
ip -br link
sudo -n true
sudo tcpdump -i eth1 -nn -c 1
```

The current code requires `eth1` and `tcpdump` privileges.

IPsec configuration or initiation failure

Run on both VMs:

```
sudo systemctl is-active strongswan
swanctl --list-conns
sudo swanctl --load-conns
sudo swanctl --load-creds
```

Check that both VMs use the same configuration and expected outer IPs.

IKE SA not established

```
sudo swanctl --list-sas
sudo journalctl -u strongswan --no-pager -n 80
```

Also verify UDP ports `500` and `4500` are allowed between the VMs.

Vite reports Unused args: 5173

Use:

```
npm run dev -- --host 127.0.0.1 --port=5173
```

Backend port 8000 is busy

```
Get-NetTCPConnection -LocalPort 8000 -State Listen | Select-Object OwningProcess
Stop-Process -Id <PID> -Force
```

PowerShell asks for a Uri when using curl

Use `curl.exe` explicitly:

```
curl.exe -sS http://127.0.0.1:8000/health
```

Analysis times out

Check health first:

```
curl.exe -sS --max-time 5 http://127.0.0.1:8000/health
```

If health is blocked, restart Uvicorn. If health works, retry analysis with `--max-time 120` and watch the backend terminal.

17. Final success checklist

- [] VM1 is running at `192.168.160.128`.
- [] VM2 is running at `192.168.160.129`.
- [] Ping succeeds in both directions.
- [] strongSwan is active on both VMs.
- [] VM1 and VM2 agents are listening on TCP `9000`.

- [] VM1 can connect to both agent ports.
- [] `eth1` is the experiment interface.
- [] `sudo -n true` succeeds on both VMs.
- [] Backend health returns `{"status": "ok"}`.
- [] Frontend opens at `http://127.0.0.1:5173`.
- [] The capture script completes all six stages.
- [] `captures/live_capture.pcap` exists on VM1.
- [] `tcpdump -r` prints packets.
- [] The PCAP has been copied to Windows.
- [] The analyzer accepts the PCAP.
- [] The analysis or report has been saved.

18. Source files used

- `testbed/sih-ipsec-analyzer_vm1/scripts/packet_capture.py`
- `testbed/sih-ipsec-analyzer_vm1/scripts/run_agent.py`
- `testbed/sih-ipsec-analyzer_vm2/scripts/run_agent.py`
- `testbed/sih-ipsec-analyzer_vm1/src/controller/controller.py`
- `testbed/sih-ipsec-analyzer_vm1/src/agent/agent.py`
- `testbed/sih-ipsec-analyzer_vm2/src/agent/agent.py`
- `testbed/sih-ipsec-analyzer_vm1/config/configuration.yaml`
- `ipsec_analyzer/backend/app/main.py`
- `ipsec_analyzer/frontend/src/services/api.js`